

What is Routing?

Routing in Flask (or any web framework) is the process of mapping a URL to a specific function in your application.

- A URL is what the user types in the browser, like /home or /user/Jani.
- A route is the rule in your Flask app that tells the server what code to run when that URL is requested.

So:

- URL → The web address the user visits.
 - Route → The Flask function that handles that URL.
-

Example:

```
from flask import Flask
app = Flask(__name__)
```

```
@app.route('/home') # This is the route
```

```
def home():         # This function handles the route
```

```
    return "Welcome Home!" # Response sent to the URL
```

- URL: http://127.0.0.1:5000/home
 - Route: /home mapped to the home() function
 - Output: "Welcome Home!"
-

Key Point:

- Routing $\neq$ URL, but routing uses URL to decide which function to execute.
- Flask “routes” each URL to a function.

1. Static Route

- A route whose **URL path is fixed** and does **not change**.
- The output is always the same for that URL.

- **Example:**

```
@app.route('/about' )
```

```
def about():
```

```
    return "This is the About Page"
```

- /about → always shows “This is the About Page”
 - You **cannot pass variables** in the URL for this route.
-

2. Dynamic Route

- A route whose URL **accepts variables** as part of the path.
- The output can **change depending on the value of the variable**.
- **Example:**

```
@app.route('/user/<name>')
```

```
def greet(name):
```

```
    return f"Hello, {name}!"
```

- /user/Jani → “Hello, Jani!”
- /user/Alex → “Hello, Alex!”

Here <name> is a **placeholder**, making the route **dynamic**.

So in short:

URL Type	Description
----------	-------------

Static Route	Fixed path, same output every time, no variables.
--------------	---

Dynamic Route	Accepts variables in URL, output changes based on input.
---------------	--

Example of static and dynamic route:

```
from flask import Flask
```

```
app=Flask(__name__)
```

```
@app.route("/")
```

```

def home():
    return 'hello world jani basha shaik from mandepudi hello'

@app.route('/<name>')

def course(name):
    return f"course name is {name}"

@app.route('/<int:a>,<int:b>')

def add(a,b):
    return f'sum of {a} and {b} is {a+b}'

if __name__=='__main__':
    app.run(debug=True)

```

♦ In Flask:

When a user requests a **URL**, Flask checks whether that URL's **path** matches any **route pattern** defined in your app.

If a match is found 👉 the **corresponding function (view function)** is executed.

example2:

dynamic routing:

```

from flask import Flask

app=Flask(__name__)

@app.route('/<name>')    # url: http://127.0.0.1:5000/jani

def welcome(name):
    return f"welcome to Flask {name}"

# addition of two numbers

@app.route('/sum/<int:a>,<int:b>,<int:c>') # url: http://127.0.0.1:5000/sum/10,20,30

def sum(a,b,c):
    print("i am sum()")
    return f"the sum of {a} and {b},{c} is {a+b+c}"

```

```
@app.route('/sum2/<int:a>,<int:b>,<int:c>') # url: http://127.0.0.1:5000/sum2/10,20,30
```

```
def sum2(a,b,c):
```

```
    print("i am sum2()")
```

```
    return f"the sum2 of {a} and {b},{c} is {a+b+c}"
```

```
@app.route('/<int:a>,<float:b>') # url: http://127.0.0.1:5000/10,2.4
```

```
def sum1(a,b):
```

```
    print("i am sum1()")
```

```
    return f"the sum of int and float is: {a+b}"
```

```
# @app.route('/<int:a>,<name>') # url: http://127.0.0.1:5000/10,jani
```

```
# def infstring(a,name):
```

```
#     return f'the a value is:{a} and name is {name}'
```

```
#start the development server:
```

```
if __name__=='__main__':
```

```
    app.run(debug=True)
```

Extra Information:

URL:

```
http://127.0.0.1:5000/jani,basha
```

1 How Flask treats this

By default, if your route is defined like this:

```
@app.route('/<name>')
```

```
def greet(name):
```

```
return f"Hello {name}"
```

- Flask sees the **entire string jani,basha as one variable**.
 - So name = "jani,basha"
 - Output: "Hello jani,basha"
-

2 To treat them as two variables

You need to **define the route with two placeholders**:

```
@app.route('/<name1>,<name2>')
```

```
def greet(name1, name2):
```

```
    return f"Hello {name1} and {name2}!"
```

- Now:
 - name1 = "jani"
 - name2 = "basha"
 - Output: "Hello jani and basha!"
-

✓ Rule of Thumb

- **Single <variable>** → Anything in that segment of the URL goes into that variable.
 - **Multiple <variable> placeholders separated by / or ,** → Each placeholder gets its own value.
-

💡 Example:

```
@app.route('/<username>/<int:age>')
```

```
def show_user(username, age):
```

```
    return f"{username} is {age} years old"
```

- URL: /jani/20 → "jani is 20 years old"
 - /jani,basha → won't work here because / is missing between variables.
-

So in your URL `http://127.0.0.1:5000/jani,basha`:

- If your route is / <name> → name = "jani,basha" ✓
 - If your route is / <name1>,<name2> → name1 = "jani", name2 = "basha" ✓
-

3. Flask redirect() Method

Definition:

redirect() is used to **redirect** the user from one route to another route or URL.

It sends an HTTP **302 (Found)** status code and redirects the browser.

✓ Example 1: Redirect to another route

```
from flask import Flask, redirect, url_for
```

```
app = Flask(__name__)
```

```
@app.route('/')
```

```
def home():
```

```
    return "Welcome to Home Page!"
```

```
@app.route('/admin')
```

```
def admin():
```

```
    return "Welcome Admin!"
```

```
@app.route('/user/<name>')
```

```
def user(name):
```

```
    if name == 'admin':
```

```
        return redirect(url_for('admin'))
```

```
    else:
```

```
return f"Hello, {name}!"
```

Examples:

- /user/admin → Redirects to /admin
 - /user/Jani → Shows "Hello, Jani!"
-

✓ Example 2: Redirect to external URL

```
@app.route('/google')
```

```
def go_to_google():
```

```
    return redirect("https://www.google.com")
```

🔗 /google → Redirects to **Google website**

🧭 4. Flask url_for() Function

Definition:

url_for() dynamically **builds a URL** for a specific function name (route) in Flask.

This is useful because if you ever **change the route name**, you don't have to update it everywhere manually.

✓ Example 1: Using url_for()

```
from flask import Flask, redirect, url_for
```

```
app = Flask(__name__)
```

```
@app.route('/')
```

```
def home():
```

```
    return "This is Home!"
```

```
@app.route('/login')
```

```
def login():
```

```
return "This is Login Page"
```

```
@app.route('/goto')
```

```
def goto():
```

```
    # redirects to the login page using function name
```

```
    return redirect(url_for('login'))
```

📌 /goto → Redirects to /login automatically
(even if you later rename /login route to /signin)

✅ Example 2: Using url_for() with parameters

```
@app.route('/profile/<username>')
```

```
def profile(username):
```

```
    return f"Welcome, {username}"
```

```
@app.route('/login_user/<username>')
```

```
def login_user(username):
```

```
    # redirects dynamically to the profile page
```

```
    return redirect(url_for('profile', username=username))
```

📌 /login_user/Jani → Redirects to /profile/Jani

✅ Example 3: url_for() in HTML templates

In your Flask template (index.html):

```
<a href="{{ url_for('home') }}">Home</a>
```

```
<a href="{{ url_for('login') }}">Login</a>
```

📌 Flask automatically converts them into:

```
<a href="/">Home</a>
```

```
<a href="/login">Login</a>
```

Summary Table

Concept	Description	Example
Dynamic Routing	Routes that accept variables	<code>/user/<name></code>
Static Routing	Routes with fixed paths	<code>/about</code>
redirect()	Redirects to another route or URL	<code>redirect(url_for('home'))</code>
url_for()	Dynamically builds URLs using route names	<code>url_for('login', username='Jani')</code>